WhatsApp NLP & Conversation Analytics Dashboard

A privacy-aware pipeline and interactive dashboard for describing the language and activity structure of exported WhatsApp chats

MSc Data Science: Final Project Report

Author: Abhishek Vanahalli

Programme: MSc Data Science

University: CHRIST (Deemed to be University)

Supervisor: Dr. Himani Sivaraman

Designation: Faculty, CHRIST (Deemed to be University)

Submission date: 10/10/2026

Code repository: https://github.com/abhishekvanahalli-bit/whatsapp-nlp-conversation-analytics

Abstract

WhatsApp chat exports are semi-structured text files that mix messages, system events, media placeholders, several scripts and personal data. Common chat analysers count words and senders without stating what the numbers mean or how personal data are protected. This project builds a reproducible pipeline and a Streamlit dashboard that describe one exported chat at a time and attach a definition, a denominator and a limitation to every result.

The pipeline parses a precisely defined export format, classifies every row (user, system, header; text, media, deleted), and applies a documented, non-aggressive preprocessing step with Unicode-aware tokenisation. It then computes unigram, bigram and trigram counts over all messages and over unique texts, TF-IDF with a repetition-sensitivity variant, Key Terms ranked by the number of distinct messages containing a term, Script Mix (the writing systems used in each message, explicitly not language identification), and conversation analytics (composition, daily activity, repetition templates) with 28 reconciliation checks that must pass before any result is written. A display-privacy layer masks phone numbers and links, pseudonymises participants and hides rare terms. Sentiment, emotion, named-entity recognition and language identification were deliberately excluded after a feasibility study found no verified model covering English, Hindi, Marathi, romanised text and code-mixing together.

The system was tested with 304 automated tests on fictional one-to-one, 12-participant and 30-participant chats, a single-sender chat and edge cases, and the pipeline reproduces byte-identical outputs on the committed sample. Results describe a single chat and are not generalisable; the main limitations are the single supported export format, English-only stopwords, unstable rankings for small chats and pattern-based (not complete) privacy masking.

Keywords: WhatsApp, natural language processing, TF-IDF, n-grams, conversation analytics, privacy, code-mixed text, Streamlit.

1. Introduction

1.1 Background and motivation

WhatsApp is a widely used messaging service, and users can export a conversation as a plain text file. These files are attractive for analysis, but they are not clean datasets: a single file contains user messages, system notices (members added, security codes changed), placeholders for media and deleted messages, multi-line messages, emoji, links and phone numbers, and often more than one script. In the chats that motivated this project, English, Marathi in Devanagari script and romanised text appeared together, sometimes inside one message.

1.2 Problem statement

Typical WhatsApp analysers produce word clouds and sender rankings without checking whether the counts mean what they appear to mean. Repeated announcements inflate word counts; media and deleted rows are not text; a script is not a language; and a chat export contains personal data that must not be displayed. The problem addressed here is: what can be described reliably about one exported chat, what must be left out or qualified, and how can this be done without exposing personal data?

1.3 Aim and objectives

The aim is to build a reproducible, tested and honest analytics system for exported WhatsApp chats. The objectives are:

- Parse an export robustly (multi-line messages, system rows, placeholders, invisible Unicode marks) and validate the result.
- Describe vocabulary with n-grams and TF-IDF while treating repeated messages explicitly instead of hiding their effect.
- Describe conversation structure (composition, activity, repetition) with counts that reconcile with each other.
- Describe writing systems (Script Mix) without claiming language identification.
- Protect privacy in every displayed and downloadable table.
- Provide an interactive dashboard, and make the whole system reproducible and tested, stating its limitations.

1.4 Scope and exclusions

The project is descriptive. It analyses one chat at a time and makes no claims about people or groups. The following were intentionally excluded: sentiment analysis, emotion classification, named-entity recognition, language identification, reply networks and predictive models. A feasibility study on emotion and sentiment (documented in the repository) found no single verified model that covers English, Hindi, Marathi, romanised Hindi/Marathi and code-mixing; implementing an unvalidated model would have produced labels that could not be defended, so it is recorded as future work.

1.5 Contributions

- A parser and preprocessing layer for one precisely defined export format, with documented handling of system rows, placeholders and invisible marks.

- A repetition-aware analysis design: every n-gram is counted over all messages and over unique texts, and TF-IDF has a primary and a sensitivity variant.
- Conversation analytics whose 28 reconciliation checks must pass before any output is produced.
- A privacy architecture separating internal tables from anonymised, display-filtered tables and downloads.
- A tested, reproducible implementation with fictional fixtures and a public repository.

1.6 Structure of the report

Section 2 reviews related work. Section 3 describes the methodology, Section 4 the implementation, Section 5 the results on the fictional sample chat, and Section 6 concludes with limitations and future work.

2. Literature Review

This chapter reviews the established concepts on which the project relies, using published sources, and states for each theme what the present work takes from the literature and what it implements itself. Statements about how this project works are supported by its own documentation, not by the cited papers.

2.1 Conversational Text Analytics

Research on WhatsApp data is still limited, partly because the messages are private and end-to-end encrypted. Garimella and Tyson (2018) showed that data from public WhatsApp groups can be collected for social-science research, obtaining nearly half a million messages from 45,794 users in 178 public groups, including users' mobile numbers; their paper illustrates both the research value of such data and the privacy exposure that comes with it. Rosenfeld et al. (2018) analysed more than six million messages from over a hundred users and built models that predict demographic characteristics from usage attributes such as group size, activity frequency, response time and message length, without using the message content.

These studies differ from the present work in purpose. The first concerns data collection from public groups and the second prediction from activity patterns. This project analyses one exported chat that the user supplies, describes both its content and its activity, makes no demographic or behavioural prediction, and treats privacy protection as part of the analysis pipeline.

2.2 Text Preprocessing and Normalization

Before any counting, text must be segmented into units. Manning, Raghavan and Schütze (2008, Chapter 2) describe tokenisation, stop-word removal and normalisation as design decisions that change what is counted, not as neutral steps; for example, whether punctuation, case and apostrophes are kept changes the vocabulary. Chat text adds further difficulty: it is informal, multilingual and mixed with placeholders, links and emoji.

The present project applies these ideas in a deliberately limited way: lower-casing, whitespace tokenisation with Unicode-aware punctuation stripping at token edges, masking of links and phone numbers, and an English-only stopword list. It does not stem, lemmatise or correct spelling, because no

validated tool for the mixed English, Marathi and romanised text of the motivating chats was adopted. This choice favours explainability, and the cost (English-only stopwords, separate word forms) is stated as a limitation.

2.3 N-gram Analysis

N-grams, contiguous sequences of n tokens, are a basic representation in statistical language processing. Brown et al. (1992), for example, build class-based n-gram language models, which show that counts of short word sequences carry useful statistical information about language. In language modelling the counts feed probability estimates; in this project they are used only descriptively, as counts of words and phrases.

The project's implementation choices follow from the data. N-grams are generated within each message, never across message boundaries, and every count is reported over all messages and over unique texts, because repeated announcements are common in group chats and inflate counts. The cited literature establishes the n-gram idea; the all-versus-unique reporting is a design decision of this project, documented in its design notes.

2.4 TF-IDF and Term Weighting

Spärck Jones (1972) gave a statistical interpretation of term specificity: a term that occurs in few documents is a better discriminator than one that occurs in many, which is the basis of inverse document frequency. Salton and Buckley (1988) compared term-weighting approaches for automatic text retrieval and established the combination of a within-document term frequency with a collection-level factor, with document-length normalisation, as a standard family of weightings. Manning et al. (2008) present the common variants, including smoothed inverse document frequency and length normalisation, and the scikit-learn library (Pedregosa et al., 2011) provides a widely used implementation.

TF-IDF was developed for retrieval from document collections. Here it is applied to a different setting, one chat in which each message is a document, so the collection is small and often repetitive. The project therefore implements the weighting itself, with the formula and normalisation documented in its design notes, compares it with the scikit-learn reference, and interprets scores only as distinctiveness within the chat. A second variant that counts each distinct text once is provided because repetition changes document frequencies.

2.5 Unicode and Mixed-Script Text

The Unicode Standard assigns every character to a script through the Script property (Unicode Consortium, n.d.). This is a property of characters, not of languages: several languages can share one script, and one language can be written in more than one. The distinction matters for the chats that motivated this project, in which English, Marathi in Devanagari script and romanised text can appear together. Barman et al. (2014) describe code-mixing in social-media text as a challenge for language identification, because the language can change within a message and the text is informal and often romanised.

For this reason the project does not attempt language identification. Its Script Mix analysis uses only the script of each letter to describe which writing systems appear in a message, an observation about characters that avoids a language label the project could not validate. The exclusion of sentiment,

emotion and entity recognition rests on the same consideration: a feasibility study documented in the repository found no single verified model covering the required combination of languages and code-mixing.

2.6 Conversation Analytics

The studies in Section 2.1 treat message and activity attributes, such as message counts, group size, frequency and response time, as informative features. Descriptive conversation analytics in this project are closer to data profiling than to modelling: they count the types of row in an export (user, system, text, media, deleted), the activity per calendar date, and the repetition of messages. The project adds internal consistency checks, 28 reconciliation checks that must all pass before any result is produced, so that the descriptive tables cannot contradict each other. No prediction, per-participant ranking or network analysis is performed.

2.7 Privacy and Anonymization

Chat exports contain personal data. The General Data Protection Regulation defines pseudonymisation as processing personal data so that it can no longer be attributed to a specific person without additional information that is kept separately (Regulation (EU) 2016/679, Article 4(5)). Replacing names with labels is therefore only one protective measure. Sweeney (2002) showed the limits of removing explicit identifiers by formalising k-anonymity, under which each person must be indistinguishable from at least k-1 others, and Narayanan and Shmatikov (2008) demonstrated that individuals can be re-identified in a sparse dataset from a small amount of auxiliary information even after names were removed.

This project applies pseudonymisation and masking in a limited, pattern-based way: participants appear as Participant_N, phone numbers and links are masked, system-message text is reduced to generic categories and terms occurring in only one message are hidden in the dashboard. These measures reduce exposure but do not provide a formal guarantee such as k-anonymity, and they are not a complete detector of personal data; a name used in several messages is still displayed. The report states this limit wherever privacy is discussed.

2.8 Research Gap and Motivation for the Present Work

The literature reviewed establishes the individual components used here: n-gram counts, TF-IDF weighting, script-based handling of multilingual text, descriptive activity analysis and the need for privacy protection of chat data. The reviewed WhatsApp studies address data collection and demographic prediction, and the privacy literature warns that removing names is not sufficient. A practical gap addressed here is the absence of a small, reproducible and documented tool that a student or analyst can run on one exported chat to obtain descriptive results with their definitions, denominators and limitations, without exposing personal data. The present work combines preprocessing, repetition-aware n-grams and TF-IDF, script-mix analysis, key terms, conversation analytics, privacy-conscious display and an interactive dashboard in one tested pipeline. This is an engineering and methodological contribution; the report does not claim a scientific novelty beyond it, and no systematic survey of existing chat-analysis tools was carried out.

3. Methodology

The methodology follows the implemented pipeline exactly. Input is validated, parsed and preprocessed; n-gram analysis, TF-IDF, Script Mix, Key Terms and conversation analytics are computed from the same text corpus; a privacy layer filters what is displayed or downloaded; and the dashboard visualises the filtered tables.

The end-to-end flow is shown in Figure 2 (Section 4.1): upload, validation, parsing, preprocessing and classification, the text analyses, conversation analytics, privacy filtering, the results bundle and the dashboard.

3.1 Input and validation

The system accepts a UTF-8 .txt export in a single supported format: [M/D/YY, H:MM:SS AM/PM] Sender: message, with system lines written as [timestamp] - text or [timestamp] text. Dates are read as Month/Day/Year and timestamps are used as recorded, because the time zone of an export is unknown. Files are rejected, with a specific message, when they are the wrong type, empty, too large, not UTF-8, in an unsupported format, or when the dates indicate a Day/Month export that could be misread. Android exports without brackets, 24-hour clocks, missing seconds and four-digit years are explicitly unsupported rather than guessed. When every month/day value is 12 or below the date order cannot be confirmed, and a warning says so.

3.2 Parsing and row classification

Every line is assigned to exactly one row: header (an untimed first line), system (timestamped lines without a participant), or user (Sender: message). Lines that do not start a new timestamp continue the previous message, so multi-line messages and blank lines inside messages are preserved. Invisible direction marks before the opening bracket, common in some exports, are ignored when detecting a new row. A part before the colon that reads like a system sentence (a system verb plus several words or a quotation mark) is not treated as a participant name.

User rows are then classified as text, media or deleted. Bracketed placeholders such as <image omitted> or <attached: ...> mark media wherever they occur; bare wording such as 'sticker omitted' or 'You deleted this message' is a placeholder only when it is the whole message, so ordinary sentences that mention these words remain text. Placeholder rows never enter the text corpus.

3.3 Preprocessing

Preprocessing is deliberately non-aggressive and explainable. Invisible marks are removed, whitespace is collapsed and text is lower-cased. Tokenisation splits on whitespace, normalises curly quotes and apostrophes, and strips punctuation of any Unicode category from token edges (including the Devanagari danda), while preserving letters, digits, emoji and inner punctuation such as don't. Links, e-mail and web addresses are replaced by a <URL> placeholder and phone numbers by <PHONE> before any term is formed. The leading [Forwarded] marker and a trailing edited marker are treated as metadata, not vocabulary. Stopword removal uses an English-only list; no Hindi or Marathi stopword list is used because no validated resource was adopted, and this is reported as a limitation.

3.4 N-gram analysis

Unigrams (with stopwords removed), bigrams and trigrams are generated per message and never across message boundaries. Each n-gram is counted over all messages and over unique texts (each distinct text counted once). The ratio of the two shows how much repetition inflates a count. Unigrams are also provided with stopwords for comparison.

3.5 TF-IDF

Each text message is one document. With N documents and document frequency df(t), idf(t) = ln((1+N)/(1+df(t))) + 1 and the weight is tf(t,d) x idf(t), L2-normalised per non-empty document. Empty documents stay in N. The primary variant keeps all messages; the sensitivity variant keeps one copy of each distinct text, showing how rankings change when repetition is removed. A top-terms table reports total and mean normalised scores. The implementation was cross-checked against scikit-learn (Pedregosa et al., 2011) and agrees to within 1e-12. A leave-one-out stability diagnostic recomputes the top terms with each document removed; it is exact up to 200 documents and uses 60 evenly spaced documents above that, with the mode recorded. TF-IDF is interpreted only as distinctiveness within one chat.

3.6 Script Mix

Each alphabetic token is assigned a script from the Unicode names of its letters (Latin, Devanagari, other, or a mixed token). A message is classified as Latin-only, Devanagari-only, other-script-only, mixed-script (more than one writing system) or no-alphabetic (for example a link-only message); the number of adjacent token pairs whose scripts differ is recorded. Script Mix is a description of writing systems. It does not identify languages: Devanagari may be Marathi or Hindi, and Latin may be English or romanised text.

3.7 Key Terms and conversation analytics

Key Terms rank terms by the number of distinct messages that contain them (coverage), alongside total occurrences (frequency), so a term repeated inside one message or only in one repeated message is not mistaken for a widely used one. Conversation analytics produce the row composition, a zero-filled daily activity calendar, hour and weekday tables, text-length summaries, counts of forwarded, media, deleted and link-containing messages, and repetition analysis. Repeated messages are grouped into anonymous templates (T1, T2, ...) by identical text after link and phone masking. Twenty-eight reconciliation checks, for example 'text + media + deleted = user rows' and 'unique + repeats = text messages', must all pass before any output is written; if one fails, nothing is produced.

3.8 Privacy methodology

Privacy is handled in layers. Phone numbers (international and local formats), links, e-mail and web addresses are masked before tokenisation. Participants are pseudonymised as Participant_N by order of first appearance within a run, with no stored mapping. System-message text is reduced to four generic categories. Internal tables that contain message text or terms are never loaded by the dashboard. A display filter hides terms that occur in only one message and any term that looks like a link, address or long digit sequence. This is a precautionary layer and is not a complete personal-data detector: a name used in several messages is still displayed.

3.9 Evaluation approach

There is no ground-truth labelling for descriptive counts, so correctness is established by construction and by checks: hand-computed unit tests, independent recomputation (Key Terms coverage, scikit-learn cross-check), reconciliation checks, determinism tests, fictional fixtures covering one-to-one, group, single-sender, zero-text, numeric, Unicode, media/deleted and privacy cases, and regression tests for every bug found in a final audit.

4. Implementation

Figure 1 shows the Overview page of the finished system, loaded with the fictional sample chat. Figure 2 shows the pipeline that produces it.

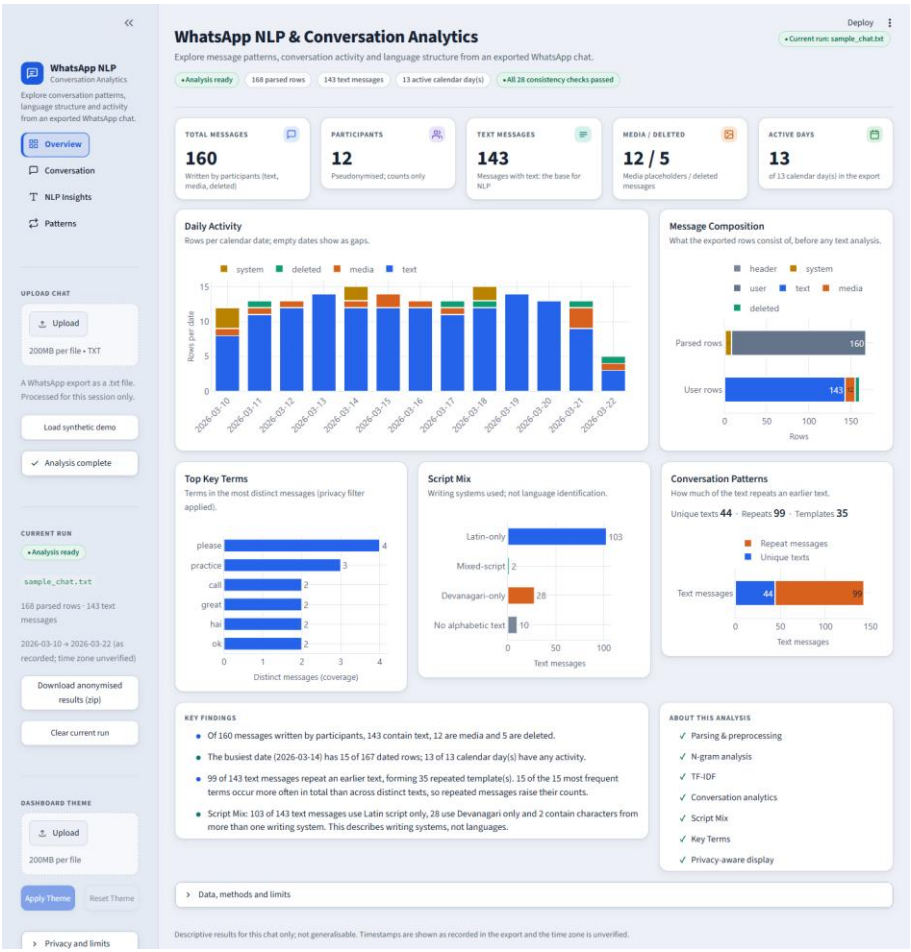


Figure 1. Overview page of the dashboard, loaded with the fictional sample chat (data/sample_chat.txt, 12 invented participants).

4.1 Architecture

Figure 2 summarises the end-to-end pipeline. A WhatsApp export is validated, parsed into header, system and user rows, preprocessed and classified as text, media or deleted. The text analyses (n-grams, TF-IDF, Script Mix and Key Terms) are computed from the same message corpus, with Key Terms reusing the n-gram and TF-IDF tables. Conversation analytics describe the whole export and run the

reconciliation checks. A privacy and display filter then produces the anonymised results bundle that the Streamlit dashboard shows, and the whole chain is evaluated by the validation described in Section 5.4. The diagram is simplified: conversation analytics and the text analyses are independent computations on the same parsed data.

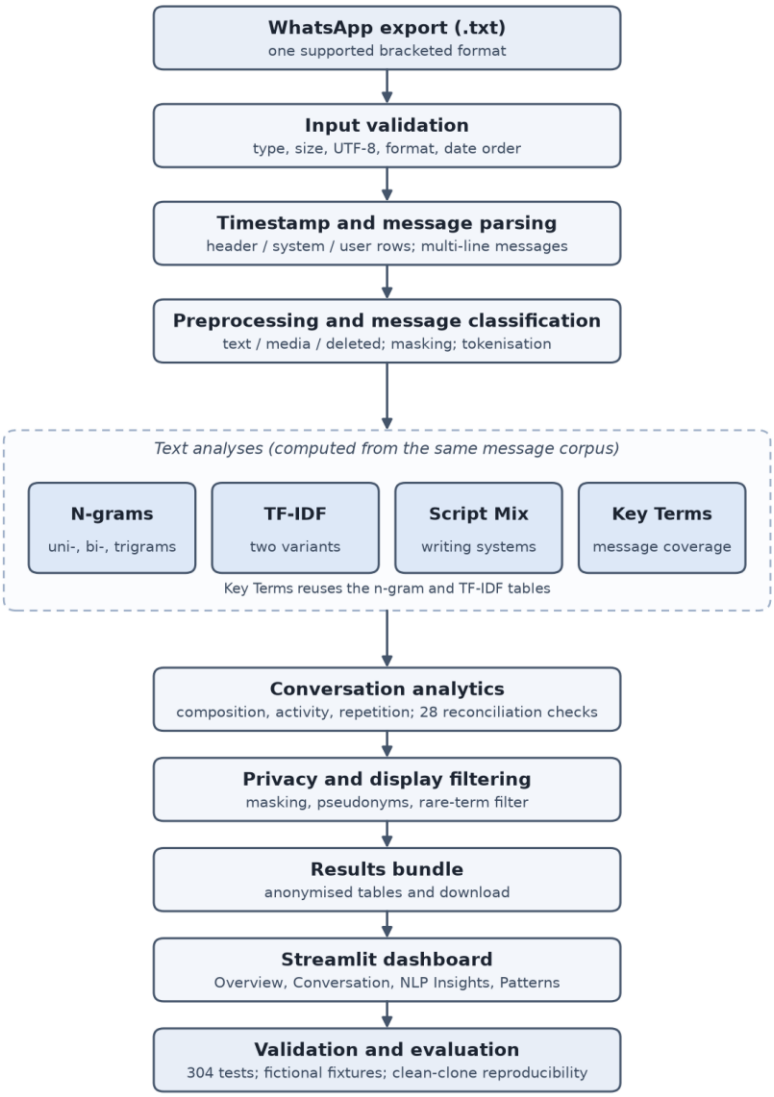


Figure 2. End-to-end methodology and implementation pipeline.

The system is written in Python 3.12 with pandas (McKinney, 2010), NumPy, Plotly and Streamlit; Pillow supports the optional theme image. The analysis modules read a processed table and write result files. For uploads, the dashboard runs the same modules inside a per-upload temporary directory, loads only an allow-list of anonymised tables and deletes the directory afterwards; only the current successful run is kept in memory, and a failed second upload leaves the previous run on screen.

Component	Responsibility
src/preprocessor.py	Parsing, row classification, placeholders, tokenisation
src/ngram_analysis.py, tfidf_analysis.py	N-grams with masking; TF-IDF and stability diagnostic
src/script_mix_analysis.py, key_terms.py	Script Mix; Key Terms
src/conversation_analytics.py	Composition, activity, repetition, 28 reconciliation checks
src/run_pipeline.py, run_manager.py	Validation, temporary run, session handling
src/display_privacy.py, results_bundle.py	Privacy filter; anonymised download
app/	Streamlit dashboard (pages, charts, theme)
scripts/run_full_pipeline.py	Reproducible command-line run

4.2 Dashboard

The dashboard has four pages. Overview summarises the chat with key figures, daily activity, message composition, top key terms, script mix, repetition and key findings built only from the run's own tables. Conversation shows activity, composition, repetition and hour, weekday and length statistics. NLP Insights has six tabs (Unigrams, Bigrams, Trigrams, TF-IDF, Script Mix, Key Terms), each with a short explanation, a chart and a collapsed supporting table. Patterns shows repetition and the most repeated anonymous templates. Before a chat is loaded, the dashboard shows an empty skeleton with a clear upload prompt. Figure 1, at the start of this section, shows the Overview page for the fictional sample chat.

4.3 Theme personalisation

An optional image can restyle the dashboard accents. A palette is derived from the image with Pillow, accents are adjusted until they meet a minimum contrast, and grey, very dark or very light images fall back to the default palette. The image is held in the browser session only and cannot affect any result or download.

4.4 Reproducibility and repository

The repository contains the source code, tests, documentation, a fictional sample chat and the sample outputs. Running scripts/run_full_pipeline.py on the sample produces 26 output files that are byte-identical across runs. Real chat exports are excluded by .gitignore and a pre-commit scan script checks the files to be committed.

4.5 Testing

The public repository contains 304 automated tests, all passing in a clean environment (302 plus two scikit-learn cross-checks that skip when scikit-learn is absent; the development environment also runs four local-only tests against a private chat). Tests cover module behaviour, parser variants, tokenisation, numeric-term typing, privacy masking, the run manager, every dashboard page for every fixture chat, and the theme system. A final audit found and fixed problems in sender-less system lines, invisible marks, placeholder classification, numeric terms, tokenisation, privacy masking, date-order handling and TF-IDF stability cost; each has a regression test.

5. Results and Analysis

All results in this section are produced from the fictional sample chat. The sample was generated with deliberately repeated messages, so it demonstrates what each analysis reports; it supports no claim about real chats. Each result is separated into observation, interpretation and limitation.

5.1 Message Composition and Activity

Observation (Figure 1): the chat has 168 parsed rows: 1 header, 7 system and 160 user rows. Of the user rows, 143 are text, 12 are media and 5 are deleted. There are 12 participants and activity on all 13 calendar days, with a peak of 15 rows on 2026-03-14.

Interpretation: most rows are analysable text, so the NLP results rest on 143 messages; media and deleted rows are counted but carry no text. Limitation: timestamps are as recorded and the time zone is unknown; with 13 days no trend can be claimed.

5.2 Repetition

Observation: 99 of the 143 text messages repeat an earlier text, leaving 44 unique texts grouped into 35 repeated templates; the largest template occurs 10 times. Interpretation: repetition strongly affects word counts, which is why every n-gram is reported over all messages and over unique texts. Limitation: only identical or mask-identical text is detected; similar but not identical messages are not.

5.3 NLP Results

This section reports what the NLP analyses produced on the fictional sample chat (143 text messages, 44 unique texts after masking). The tables are taken from the committed files in results/sample/. Because the sample was generated with deliberately repeated messages, the results show how each analysis behaves; they are not findings about real conversations.

5.3.1 N-gram analysis

Observation: The unigram table (English stopwords removed) contains 126 distinct terms, the bigram table 149 and the trigram table 111. Table 1 lists the most frequent terms. The most frequent unigrams are 'will' (14 messages), 'ok' (12), 'call' (11) and 'join', 'please' (10 each). The placeholder <URL> appears 9 times, because links are replaced by a placeholder rather than deleted. Among the bigrams and trigrams, 'join the', 'ok will', 'the call' and 'will join' (bigrams) and 'join the call', 'ok will join' and 'will join the' (trigrams) each occur 10 times. 'details here <URL>' occurs 9 times, and the Devanagari phrases 'कल मैच' and 'कल मैच है' occur 7 times.

Table 1. Most frequent n-grams in the fictional sample (count over all messages / count over unique texts).

Type	1st	2nd	3rd	4th	5th
Unigram	will 14 / 2	ok 12 / 2	call 11 / 2	join 10 / 1	please 10 / 4
Bigram	join the 10 / 1	ok will 10 / 1	the call 10 / 1	will join 10 / 1	details here 9 / 1
Trigram	join the call 10 / 1	ok will join 10 / 1	will join the 10 / 1	details here <URL> 9 / 1	कल मैच है 7 / 1

Interpretation: The second number in each pair is the count over unique texts. For 'join the call', 'ok will join' and 'will join the' it is 1, while the all-message count is 10; this pattern is consistent with a single five-word message ('ok will join the call') that was posted ten times, and the same holds for 'details here <URL>' (9 / 1) and the Devanagari phrase (7 / 1). In the unigram table, 102 of the 126 terms have a larger all-message count than unique-text count, so repetition affects most of the vocabulary. The results therefore indicate that a high frequency in this sample mostly reflects repeated messages and not wide lexical use. This is the reason every n-gram is reported both ways. N-grams are formed inside each message and never across message boundaries, so a phrase such as 'will join the' cannot be an artefact of two neighbouring messages being joined.

Limitation: Frequency shows that a sequence occurs, not what it means, and nothing in these counts establishes topic, intent or importance. The sample is small and artificially repetitive, English stopword removal removes 31.9 % of Latin tokens (526 to 358) but none of the 86 Devanagari tokens, so Devanagari function words are not filtered and can rank highly. The finding should be interpreted cautiously for that reason.

5.3.2 TF-IDF analysis

Why TF-IDF was used. Raw frequency rewards terms that are simply common. TF-IDF weights a term by how many messages contain it, so a term found in few messages receives a higher weight than one found in most. Each text message is one document. The implemented weight is tf(t,d) x idf(t) with idf(t) = ln((1 + N) / (1 + df(t))) + 1, followed by L2 normalisation of each non-empty document vector. N counts every document, including empty ones. Documents with no retained terms contribute no term weights; in the sample there are none (0 empty documents in either variant). Placeholders for links and phone numbers are kept as per-message metadata and in the n-gram tables, but they are excluded from the ranked TF-IDF vocabulary.

Observation: The primary variant (all 143 messages) has a vocabulary of 124 terms; the sensitivity variant (44 unique texts) has the same 124 terms. Table 2 shows the highest-ranked terms by total normalised score in each variant.

Table 2. Highest total normalised TF-IDF scores in the fictional sample (df = number of messages containing the term).

Variant	Term	Document frequency and total score	Mean score in messages containing the term
All messages (N = 143)	details	df 9, total 9.00	mean 1.00
	will	df 14, total 6.34	mean 0.45
	ok	df 12, total 5.63	mean 0.47
	call	df 11, total 5.63	mean 0.51
Unique texts (N = 44)	please	df 4, total 1.62	mean 0.41
	practice	df 3, total 1.34	mean 0.45
	call	df 2, total 1.16	mean 0.58

Interpretation: 'details' ranks first in the primary variant with a total of 9.00, equal to its document frequency, and a mean normalised score of 1.00. A normalised score of 1.00 means that in each of the nine messages containing it, 'details' is the only retained term (the rest of the message, 'here <URL>', is

removed as a stopword or placeholder). Its rank is therefore driven by the repeated message 'details here <URL>' and not by the word being informative. 'will' ranks second because it occurs in 14 messages; its normalised weight within each message is lower (mean 0.45) because those messages contain several other terms. This illustrates how total TF-IDF differs from raw frequency: the two rankings are related but not identical, and the ranking depends on message length as well as on how many messages contain a term. In the sensitivity variant, where each distinct text is counted once, the ranking changes: 'please' and 'practice' lead and 'details' falls to a document frequency of 1. The overlap between the two variants' top-ten term lists is 0.4 (four of ten), which shows how strongly repetition shapes the primary ranking. The stability diagnostic supports a cautious reading: with each document removed in turn, the top-ten list keeps on average 97.5 % of its terms in the primary variant (96.4 % in the sensitivity variant), with a minimum overlap of 80 % in both.

Limitation: A high TF-IDF score means distinctive within this chat, not important in general. Many terms share the same document frequency and therefore the same IDF, so ties are common, and rankings on a corpus of this size are not stable evidence about language use. No single term is claimed to be best.

5.3.3 Script Mix analysis

Script Mix is a writing-system analysis, not language identification. Each alphabetic token is assigned a script from the Unicode names of its letters, and each message is placed in one class.

Observation: Table 3 gives the class counts. Of the 143 text messages, 103 use Latin script only, 28 use Devanagari only, 2 are mixed-script and 10 contain no alphabetic token (for example a message consisting only of a link or an emoji). No message is classed as another script. Among the 612 alphabetic tokens, 526 are Latin and 86 are Devanagari. The two mixed-script messages each contain one change of script between adjacent tokens, and Devanagari makes up about 20 % of their alphabetic tokens. Counting each distinct text once gives 31 Latin-only, 8 Devanagari-only, 1 mixed-script and 4 no-alphabetic messages out of 44.

Table 3. Script Mix classes in the fictional sample.

Class	All messages	Unique texts
Latin-only	103	31
Devanagari-only	28	8
Mixed-script	2	1
Other script only	0	0
No alphabetic text	10	4
Total text messages	143	44

Interpretation: In this sample the writing systems are largely separate: messages are written in Latin or in Devanagari, and only a very small number combine both. The repeated messages inflate the Latin-only and Devanagari-only counts (103 against 31 and 28 against 8), which is why both variants are shown. Messages in the no-alphabetic class carry no script information and should not be read as a language.

Limitation: These categories describe characters, not languages. A Latin-only message may be English or romanised Hindi or Marathi, and a Devanagari-only message may be Hindi or Marathi, so the results do not show that any message is English, Hindi or Marathi, and they do not show code-switching. A mixed-

script message is only an observation that two writing systems appear in it. The sample is fictional and the mixed-script group contains two messages, so no pattern about mixing can be inferred.

5.3.4 Key Terms

Key Terms are ranked by coverage: the number of distinct (unique) messages that contain a term, with ties broken alphabetically. The table also reports the number of messages containing the term when repeats are included and the total number of occurrences. The internal table contains 124 terms; the dashboard shows these after its privacy filter, which hides terms occurring in only one message.

Table 4. Highest-ranked Key Terms in the fictional sample.

Term	Distinct messages containing the term (coverage)	Messages containing the term, repeats included	Total occurrences
please	4	10	10
practice	3	7	7
call	2	11	11
great	2	8	8
ok	2	12	12
see	2	8	8

Observation: 'please' ranks first with coverage 4 and 'practice' second with coverage 3. Several terms with high raw frequency rank lower: 'ok' occurs 12 times and 'call' 11 times, but each appears in only 2 distinct messages.

Interpretation: Coverage separates terms that are spread across different messages from terms that are frequent because one message was repeated. In this sample, 'ok' and 'call' are frequent but their coverage is low, which is consistent with the repetition found in Section 5.2; 'please' is present in more distinct messages. Ranking by coverage therefore avoids overemphasising a term that belongs to a single repeated message.

Limitation: Coverage values are small (at most 4 of 44 unique texts), so the ordering among terms with equal coverage is decided alphabetically and carries no meaning. The method inherits the English-only stopword list and does not group word forms, and a ranking of existing counts is not a topic model.

5.4 Validation and Reproducibility

The results above are only useful if the software that produced them is correct and repeatable. This section summarises the evidence gathered, and states what it does and does not establish. All validation used fictional or public data.

5.4.1 Automated testing

Observation: Run from a clean clone of the public repository in a new virtual environment, the test suite reports 304 passed and 0 failed. The development environment reports 308, because it includes four regression tests that run only against a private development chat; these tests are excluded from the public repository (they are git-ignored and skip when the file is absent) and are not described further here. Without scikit-learn installed, two cross-check tests skip by design.

Interpretation: The tests exercise module behaviour (hand-computed n-gram and TF-IDF cases), parser variants, tokenisation, numeric terms, privacy masking, run management and the dashboard pages. Each problem found in a final audit has a named regression test. Limitation. Passing tests show that the implemented behaviour matches what the tests specify; they do not show that the specifications cover every real export.

5.4.2 Generalisation across fictional chats

Observation: Ten fictional fixtures were run through the complete pipeline, and all completed with the 28 reconciliation checks passing (Table 5). Further cases were tested inline: one, two and five text messages; chats of identical messages, stopwords only, emoji only and numbers only; CRLF line endings; and unsupported or invalid files.

Table 5. Fictional fixtures run through the pipeline (rows parsed, text messages, participants).

Fixture	Rows	Text messages	Participants
one-to-one chat	71	66	2
12-participant group	168	143	12
30-participant group	138	122	30
single sender	33	30	1
zero text (media, deleted, system only)	5	0	2
system events	12	2	2
Unicode and invisible marks	13	11	3
media and deleted placeholders	23	5	2
numeric terms	11	10	2
privacy cases	19	18	2

Interpretation: The validation demonstrates that the pipeline does not depend on the number of participants, on a single sender, or on a chat containing text. A chat with no text messages produces a limited result with a warning and no crash, a single-sender chat runs with a small-chat warning, and numeric terms such as 007 remain strings. Day/Month dates, Android-style lines and non-UTF-8 files are rejected with specific messages instead of being misread. Limitation. The fixtures were written by the author and generated with fixed seeds, so they test the behaviours the author anticipated.

5.4.3 Parser and reconciliation validation

Parser behaviour is validated by fixtures for sender-less system lines, invisible marks before the timestamp, multi-line messages, placeholders in Android and iOS wording, and text that merely mentions a placeholder. In addition, the conversation analytics run 28 reconciliation checks, for example that text, media and deleted rows sum to user rows and that unique texts plus repeats equal the number of text messages. All outputs are written only if every check passes; in the fictional sample all 28 passed. These checks verify internal consistency, not that the export itself was complete.

5.4.4 TF-IDF validation

The custom TF-IDF implementation was compared with an independent reference, scikit-learn's TfidfVectorizer, configured with smoothed IDF and L2 normalisation. The tests require the maximum absolute difference to be below 1e-12; the largest difference recorded for the sample is about 1.1e-16 in both variants. The two implementations therefore agree to within floating-point rounding, which is not the same as exact equality. This validates the arithmetic, not the usefulness of the ranking.

5.4.5 Reproducibility

The pipeline was run on the public sample (data/sample_chat.txt) twice in clean temporary folders and the 26 output files were byte-identical across runs and identical to the committed results/sample. A fresh clone of the public repository was then installed in a new environment and the pipeline reproduced the committed outputs without any private data. The only differences were the library-version fields recorded in the TF-IDF metadata file (for example NumPy and pandas versions) and Windows line endings. The whole run takes a few seconds for the sample.

5.4.6 Dashboard validation

The public sample was loaded in the dashboard in Microsoft Edge at 1440 px (and, in earlier checks, at 1366 px). The empty state, the upload flow, the Overview, Conversation and Patterns pages and all six NLP tabs were opened, no horizontal overflow was found, and the rendered text contained none of the fictional sender names, phone numbers or links. Browser testing was manual and scripted capture of screenshots; it checks layout and absence of leaked values, not usability with real users.

5.4.7 Privacy validation

The files of the public repository were scanned for phone numbers, e-mail addresses, WhatsApp invite links and API keys or tokens; no real instances were found. The private source chat is excluded from the repository, and the public sample, fixtures and results use fictional participants and invented data. Privacy masking and the display filter are a precautionary layer, not a guarantee of complete personal-data detection: unusual number or link formats may be missed, and a name used in several messages is still displayed.

5.4.8 Limits of the validation

The validation relies on synthetic and fictional fixtures instead of a collection of real iOS and Android exports. It therefore demonstrates implementation behaviour and reproducibility, not population-level claims about WhatsApp conversations. Android and other formats without the bracketed timestamp are unsupported and remain documented limitations, and iOS-style conventions were verified only on fictional fixtures that imitate them.

5.5 Limitations

- Only one export format is supported; Android no-bracket, 24-hour, no-seconds and four-digit-year formats are not. iOS-style conventions were verified only on fictional fixtures.
- The time zone is unknown, and when all month/day values are 12 or below the date order cannot be confirmed.
- Stopword removal is English-only, so Hindi, Marathi and romanised function words can dominate rankings.

- Small chats give unstable TF-IDF and Key Terms rankings.
- Privacy masking is pattern-based; unusual phone or link formats may be missed and names inside messages are not detected.
- The TF-IDF stability diagnostic is sampled above 200 messages, and large-scale behaviour has limited validation (default limit 1,000 text messages).
- Generalisation was tested with fictional fixtures; no real group or iOS exports were available.

6. Conclusion and Future Work

6.1 Future Work

- Broader WhatsApp export-format support (for example Android and 24-hour exports) with date-order inference and fixtures for each variant.
- Improved multilingual stopword handling for Hindi, Marathi and romanised text, with a validated resource.
- Validation on larger and real group and iOS datasets.
- Stronger personal-data detection, including names.
- A more robust large-scale evaluation of TF-IDF stability.
- Language-aware analysis, if a validated model for code-mixed text becomes available.

6.2 Conclusion

This project addressed a practical problem: an exported WhatsApp chat is a messy, multilingual and personal text file, and the usual ways of summarising it do not say what the numbers mean or how personal data are protected. The work implemented a reproducible pipeline and a Streamlit dashboard that describe one exported chat at a time.

The implementation covers a parser and preprocessing layer for one precisely defined export format, repetition-aware unigram, bigram and trigram counts, TF-IDF with a sensitivity variant, Script Mix as a description of writing systems, Key Terms ranked by message coverage, conversation analytics with 28 reconciliation checks, a privacy-conscious display and download layer, and an interactive dashboard. Sentiment, emotion, entity recognition and language identification were deliberately excluded, because no validated approach for the project's code-mixed text was available.

The validation consisted of 304 automated tests in a clean clone of the public repository, ten fictional fixtures (one-to-one, 12- and 30-participant groups, single-sender, zero-text and edge cases), a comparison of the TF-IDF implementation with scikit-learn that agrees to within floating-point rounding, byte-identical reproduction of the committed sample outputs, browser checks of the dashboard, and scans of the public repository for personal data. These results show that the pipeline behaves as specified on the cases tested and that its outputs can be reproduced by someone else without private data.

They do not show more than that. The public demonstration uses a fictional dataset, so the numerical results reported in Section 5 demonstrate pipeline behaviour and reproducibility; they do not describe how people use WhatsApp in general. Only one export format is supported, Android and other formats

without bracketed timestamps are not, privacy masking is pattern-based and not a complete personal-data detector, and Script Mix is not language identification. Validation on real group and iOS exports was not possible within this project.

Within these limits, the project provides a documented, tested and reproducible implementation of a descriptive analysis pipeline for exported chats, with its assumptions and limitations stated alongside its results, which is the aim set at the start of the report.

References

Barman, U., Das, A., Wagner, J., & Foster, J. (2014). Code mixing: A challenge for language identification in the language of social media. In *Proceedings of the First Workshop on Computational Approaches to Code Switching* (pp. 13–23). Association for Computational Linguistics. https://doi.org/10.3115/v1/W14-3902

Brown, P. F., Della Pietra, V. J., deSouza, P. V., Lai, J. C., & Mercer, R. L. (1992). Class-based n-gram models of natural language. *Computational Linguistics, 18*(4), 467–480. https://aclanthology.org/J92-4003/

Garimella, K., & Tyson, G. (2018). WhatsApp, doc? A first look at WhatsApp public group data. *Proceedings of the International AAAI Conference on Web and Social Media, 12*(1). https://doi.org/10.1609/icwsm.v12i1.14989

Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to information retrieval*. Cambridge University Press. https://nlp.stanford.edu/IR-book/

McKinney, W. (2010). Data structures for statistical computing in Python. In *Proceedings of the 9th Python in Science Conference* (pp. 56–61). https://doi.org/10.25080/Majora-92bf1922-00a

Narayanan, A., & Shmatikov, V. (2008). Robust de-anonymization of large sparse datasets. In *2008 IEEE Symposium on Security and Privacy* (pp. 111–125). IEEE. https://doi.org/10.1109/SP.2008.33

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830. https://jmlr.org/papers/v12/pedregosa11a.html

Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data, and repealing Directive 95/46/EC (General Data Protection Regulation). *Official Journal of the European Union, L 119*, 1–88. https://eur-lex.europa.eu/eli/reg/2016/679/oj

Rosenfeld, A., Sina, S., Sarne, D., Avidov, O., & Kraus, S. (2018). WhatsApp usage patterns and prediction of demographic characteristics without access to message content. *Demographic Research, 39*, 647–670. https://doi.org/10.4054/DemRes.2018.39.22

Salton, G., & Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. *Information Processing & Management, 24*(5), 513–523. https://doi.org/10.1016/0306-4573(88)90021-0

Spärck Jones, K. (1972). A statistical interpretation of term specificity and its application in retrieval. *Journal of Documentation, 28*(1), 11–21. https://doi.org/10.1108/eb026526

Sweeney, L. (2002). k-anonymity: A model for protecting privacy. *International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems, 10*(5), 557–570.

Unicode Consortium. (n.d.). *Unicode Standard Annex #24: Unicode script property*. Retrieved October 2026, from https://www.unicode.org/reports/tr24/

Appendix A. Report status

Section	Status
Cover	Complete
Abstract	Complete
1 Introduction	Complete
2 Literature Review	Complete (13 verified sources, all cited in the text)
3 Methodology	Complete
4 Implementation	Complete (Figures 1 and 2)
5 Results and Analysis	Complete (5.1 to 5.5)
6 Conclusion and Future Work	Complete
References	Complete
Remaining placeholders	None